

Module 1 :- Analysis Of Basic Algorithms

* Selection Sort

→ Comparison based sorting algorithm that sorts by repeatedly selecting the smallest element from the unsorted portion & swapping it with the first unsorted element.

The algo :-

1. firstly swap the smallest element with the first element making the smallest element at the correct order.
2. Then find the smallest among remaining elements & swap it with the second element.
3. Keep doing this until all the elements are not sorted.

Eg.

arr[] =

64	25	12	22	11
----	----	----	----	----

Step 1:-

Smallest element :- 11

Swap with first element, then arr becomes

11	25	12	22	64
----	----	----	----	----

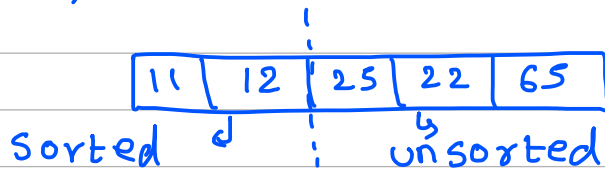
sorted[↙] ; unsorted[↘]

Step 2:-

Smallest unsorted element :- 12

Swap with the first element in the unsorted (12, 25)

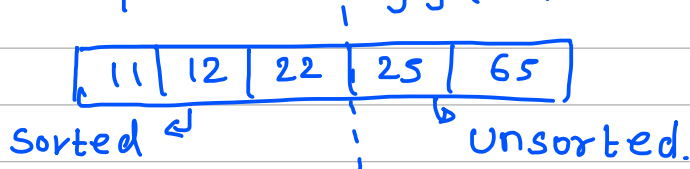
part, then arr becomes



Step 3:-

Smallest unsorted element :- 22

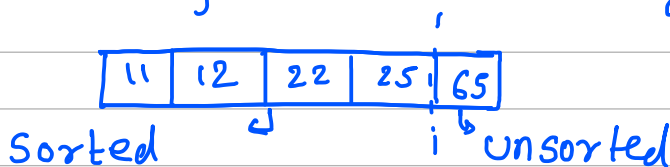
Swap accordingly (22, 25)



Step 4:-

Next smallest element:- 25

Already sorted arr stays same



Step 5:-

Last element will always be sorted

∴ Final sorted arr :-

11	12	22	25	65
----	----	----	----	----

Complexity Analysis:-

→ Time complexity:-

i) Best case $\Omega(N^2)$:-

ii) Average case $\Theta(N^2)$:-

iii) Worst case $O(N^2)$:-

Takes $n-1$ comparisons for first step, $n-2$ comparisons & so on.

$$\Rightarrow (n-1) + (n-2) + (n-3) + \dots + (1) =$$

$$\frac{n(n-1)}{2} \approx O(N^2)$$

→ Space Complexity $O(1)$. No extra space taken.

→ Applications :-

- 1) Used in small datasets where simplicity matters more than speed.
- 2) Useful in memory constrained or flash based systems as it performs minimum swaps.
- 3) Applied when only the k smallest or largest elements are needed instead of a full sorting.

* Insertion Sort

→ A simple sorting algorithm that works by iteratively inserting each element of the unsorted list to its correct position in the sorted portion of the list.

The algo :-

1. The first element is always considered to be sorted.
2. Start with the second element & compare it with the first, swap them if required.
3. Move to the third element compare it with the first two, & put it in its correct position.
4. Repeat until the entire array is sorted.

Eg.

$arr[] = \{23, 1, 10, 5, 2\}$

Step 1:- Starting with the second element

Compare 1 with 23 (current element with the sorted part)

Swap 1 < 23, insert 1 before 23.

Arr becomes $\{1, 23, 10, 5, 2\}$

Step 2:-

Compare 10 with 23 & 1

Since $10 > 1$ but $10 < 23$ insert 10 between 1 & 23.

Arr becomes $\{1, 10, 23, 5, 2\}$

Step 3:-

Compare 5 with 23, 10 & 1

Since $5 > 1$, $5 < 10$ insert 5 in between 1 & 10.

Arr becomes {1, 5, 10, 23, 2}

Step 4:-

Compare 2 with 23, 10, 5, 1

Since $2 > 1$ & $2 < 5$ insert 2 in between 1 & 5.

Arr becomes {1, 2, 5, 10, 23}

The array is now sorted. [1, 2, 5, 10, 23]

Complexity analysis :-

→ Time complexity:

- 1) Best case $\Omega(N)$:-
 - Input is already sorted array.
 - Each element is compared with its preceding element until no swaps are needed resulting in linear time complexity.
 - 2) Average case $\Theta(N^2)$:-
 - This complexity arises from the nature of algorithm, which involves pairwise comparison & swaps to sort the elements.
 - Although the exact number of comparisons and swaps may vary, the average case complexity still remains quadratic.
 - 3) Worst case $\Theta(N^2)$:-
 - Array is in reverse sorted order
 - Each element needs to be compared & possibly swapped with every preceding element, resulting in quadratic time complexity.
- Space complexity $O(1)$:- No extra space required.

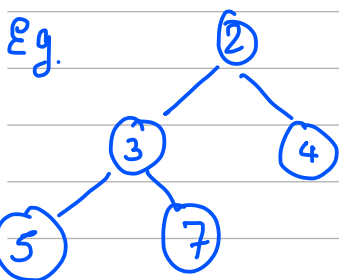
→ Applications:-

- i) Efficient for nearly sorted datasets due to its $O(N)$ best case time.
- ii) Used in online sorting, where elements arrive one by one
- iii) Used inside some hybrid algorithms for sorting small subsets.

★ Heap Sort

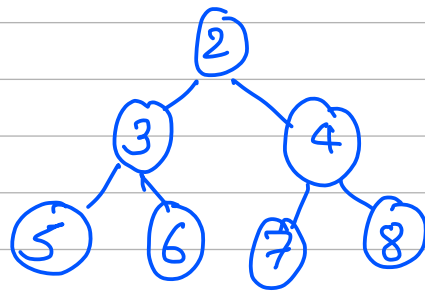
A heap is a complete binary tree data structure that satisfies the heap property: for every node, the value of its children is greater than or equal to its own value.

Heaps are usually used to implement priority queues, where the smallest (or largest) element is always at the root of the tree.



min-heap

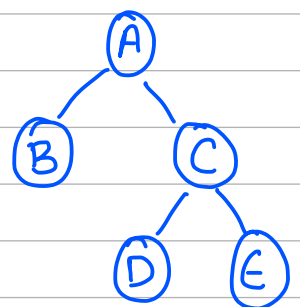
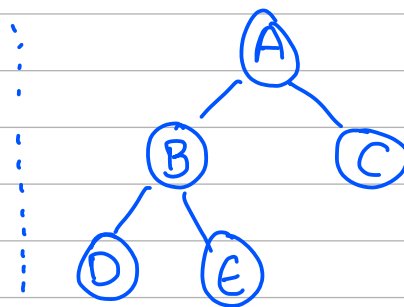
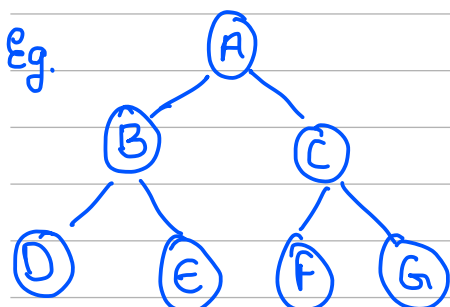
min-heap



min-heap

What is a complete binary tree?

→ For this we need to know the array representation of a complete binary tree.



This is how the relationship is preserved:-

If a node is at index - i

Its left child is at - $2*i$

Its right child is at - $2*i+1$

Its parent is at - $\lfloor \frac{i}{2} \rfloor$

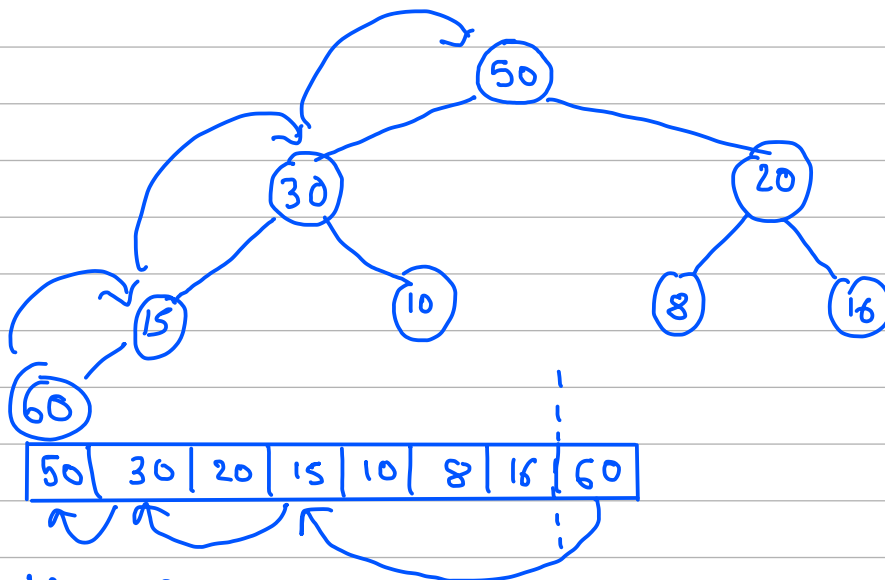
A binary tree is called a complete binary tree if:

In its array representation if there is not empty or missing element.

$\therefore$ In the above examples all the binary trees are complete binary trees except the rightmost one.

* Insertion & deletion in a heap.

→ Insertion

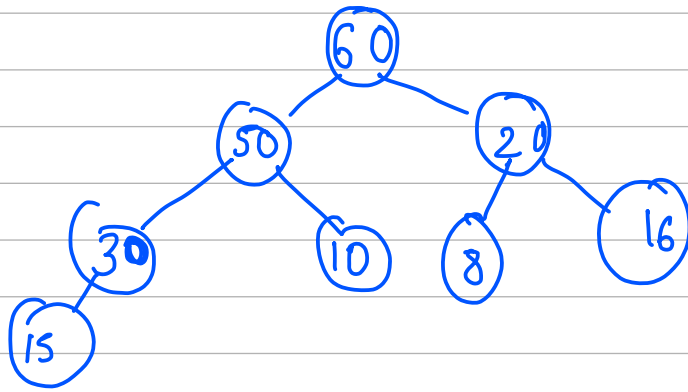


Inserting 60:-

Insert in the last of the array.

Then compare with its parents, (its parent is located on $\lfloor \frac{i}{2} \rfloor^{\text{th}}$ position). Until the root. Swap if necessary.

∴ The heap & it's corresponding array now become:-

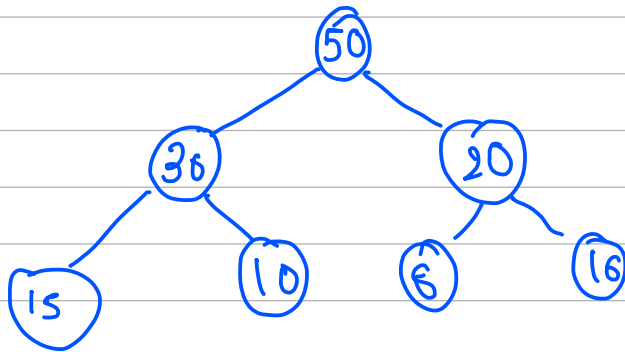


Time :-
Complexity
 $\Omega(1)$, $O(\log N)$, $O(\log N)$.

H:-

60	50	20	30	10	8	16	15
----	----	----	----	----	---	----	----

→ Deletion

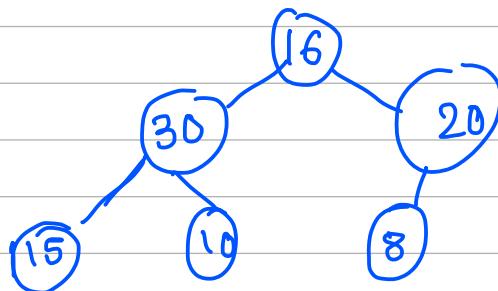


H:-

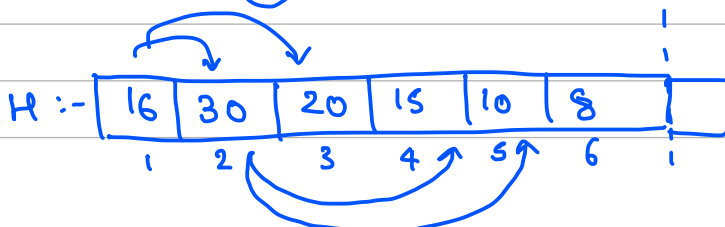
50	30	20	15	10	8	16
----	----	----	----	----	---	----

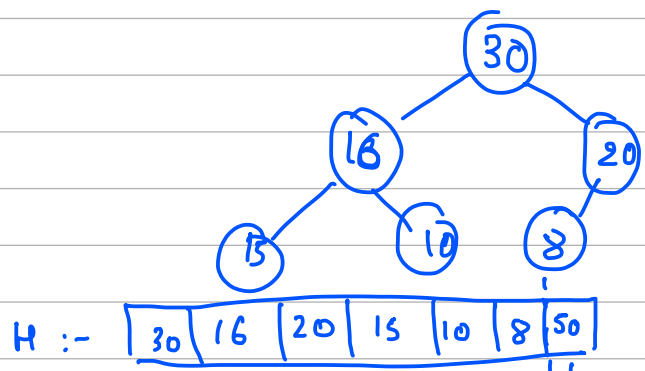
Only root is deleted always. & the last element comes in its place.

Then it is compared to its child/children & they are swapped if required. Whichever of its children is greater that will take its place. The children are present at $2*i$ & $2*i+1$.



Time :-
Complexity
 $\Omega(1)$, $O(\log N)$, $O(\log N)$.





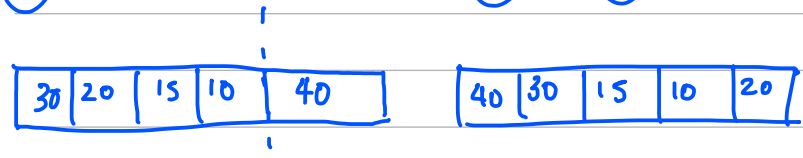
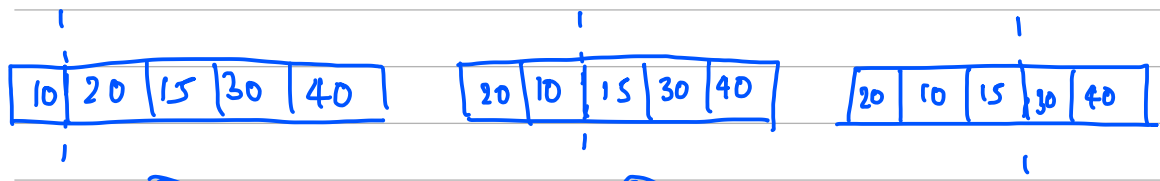
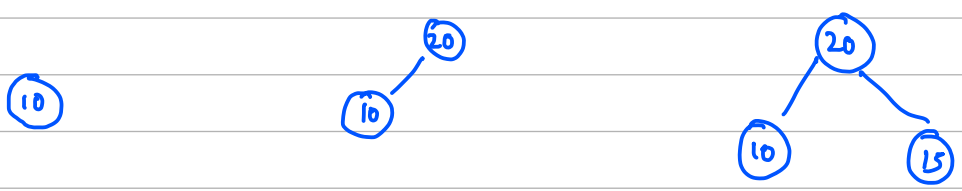
Stored in the last of the array,
not the part of heap.

* Heap Sort:-

Two step process:

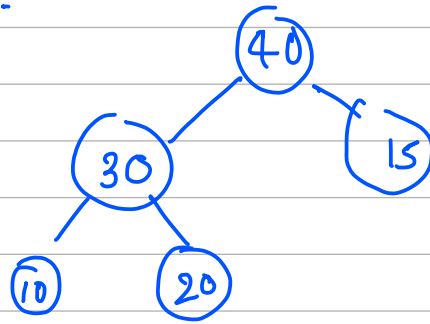
- 1) Create the heap of the given elements
- 2) The delete these elements one by one the elements will be sorted.

Eg.

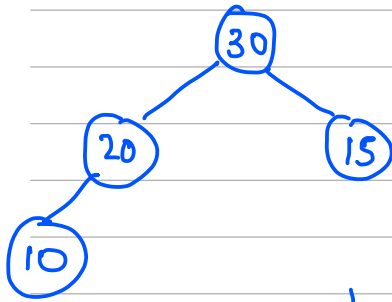


T.C upto this step:- $n * \log n$ [n elements, $\log N$ comparisons].

max heap :-

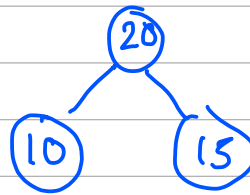


40	30	15	10	20
----	----	----	----	----

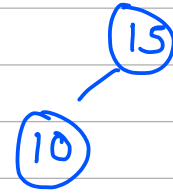


30	20	15	10	40
----	----	----	----	----

deleted element



20	10	15	30	40
----	----	----	----	----



15	10	20	30	40
----	----	----	----	----

10

10	15	20	30	40
----	----	----	----	----

Empty heap

10	15	20	30	40
----	----	----	----	----

The array is finally sorted.

Time complexity :-

Creation & insertion :- $n \log n$

Deletion :- $n \log n$

Total :- $n \log n + n \log n = 2 \log n$

$= O(\log n)$

in all three cases.

→ Applications:-

- 1) Used to priority queues in operating systems & scheduling systems.
- 2) Applied in graph algorithms like Dijkstra's algorithm for efficient minimum extraction.
- 3) Used for solving Top-k problems such as finding k largest or smallest elements.

★ Counting Sort

→ Non comparison based sorting algorithm. Useful when the range of input values is small compared to the number of elements to be sorted.

The algo:-

- i) Declare a count array $\text{cntArr}[]$ of size $\max(\text{Arr}[]) + 1$ & initialise it with 0.
- ii) Store the frequency of each element of array. i.e., $\text{cntArr}[\text{arr}[i]]++$ for $0 \leq i < N$.
- iii) Calculate the prefix sum at every index of $\text{cntArr}[]$.
- iv) Create an array $\text{ans}[]$ of size N ,
- v) Traverse array $\text{arr}[]$ from end & update
$$\text{ans}[\text{cntArr}[\text{arr}[i]] - 1] = \text{arr}[i].$$
& then update $\text{cntArr}[\text{arr}[i]] = \text{cntArr}[\text{arr}[i]] - 1$.

Step 1:-

Eg.

0	1	2	3	4	5	6	7
2	5	3	0	2	3	0	3

Max = 5

Step 2:-

0	1	2	3	4	5
0	0	0	0	0	0

Step 3:- Storing each element frequency in the cntArr[].

0	1	2	3	4	5
2	0	2	3	0	1

Step 4:- Storing the prefix sum or cumulative sum in cntArr[].

0	1	2	3	4	5
2	2	4	7	7	8

Step 5:-

Update $\text{ans}[\text{cntArr}[\text{arr}[i]] - 1] = \text{arr}[i]$ & decrement $\text{cntArr}[\text{arr}[i]]$

0	1	2	3	4	5	6	7
2	5	3	0	2	3	0	3

0	1	2	3	4	5
2	2	4	7	7	8

for $i = 0$

$$\text{ans}[\text{cntArr}[\text{arr}[0]] - 1] = \text{arr}[0]$$

$$\text{ans}[\text{cntArr}[2] - 1] = 2$$

$$\text{ans}[4 - 1] = 2$$

$$\text{ans}[3] = 2.$$

0	1	2	3	4	5	6	7
			2				

for $i = 1$.

$$\text{ans} [\text{cntArr}[\text{arr}[i]] - 1] = \text{arr}[i]$$

$$\text{ans} [7] = 5$$

Ans [] :-

0	1	2	3	4	5	6	7
			2				5

keep on doing this until $i < n$. & finally the array will be sorted,

Ans [] :-

0	0	2	2	3	3	3	5
---	---	---	---	---	---	---	---

* Asymptotic Notations :-

→ Used to describe time/space complexity of algorithms when input size $n \rightarrow$ large.

This ignores constants and lower order terms.

1) Big-O Notation (O) :- gives the upper bound (worst-case time complexity) of an algorithm.

Eg. i)

```
for (int i=0; i<n; i++) {
    cout << i;
}
```

}

Runs n times $O(N)$.

ii) $3n^2 + 5n + 7 = O(N^2)$.

2) Big- Ω Notation (Omega) :- Gives the lower bound or the best case of an algorithm,

Eg. Linear search best case $\rightarrow \Omega(1)$.

3) Big- Θ Notation (Theta) :- Gives the average time

complexity.

Eg. :- Average time complexity of merge sort :- $\Theta(n \log n)$.

* Order of growth:

As n increases, different functions grow at different speeds.

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$$

- 1) Constant time :- $O(1)$; independent of input size.
- 2) Logarithmic time :- $O(\log_2 n)$; Reduces the problem by half in each step. Eg. Binary search.
- 3) Linear time :- $O(N)$; Single loop. Eg. Linear search.
- 4) $O(n \log n)$:- Divide & conquer sorting & algos. Eg. Merge Sort, Heap sort.
- 5) Quadratic $O(N^2)$:- Nested loops. Eg. Bubble sort & selection sort.